

OPERATING SYSTEMS LABORATORY

PRACTICAL EXPERIMENT – 12

Name	ANSHU
Roll Number	2520030440
Section	8

QUESTION 12

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

AIM

To create single and multiple pipes, transfer data between connected processes, manage file descriptors, synchronize processes, and test data flow through a pipeline.

THEORY

A pipe provides one-way communication between processes. The pipe() system call creates file descriptors for reading and writing. fork() creates child processes, while read() and write() transfer data through the pipe. close() releases descriptors and wait() synchronizes parent and child processes. Multiple pipes can connect several processes in sequence.

ALGORITHM

1. Create a pipe using pipe().
2. Create a child process using fork().
3. Write data from the first process into the pipe.
4. Read the data from the receiving process.
5. Close unused file descriptors.
6. Create two pipes for a multiple-process pipeline.
7. Launch three processes using fork().
8. Transfer data from Process 1 to Process 2 and then Process 3.
9. Wait for all processes to complete.
10. Display successful completion of the pipeline tests.

C PROGRAM

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int p1[2], p2[2];
    char msg[] = "Hello Pipeline";
    char buf[50];
    pid_t a, b, c;

    printf("=====\n");
```

```

printf(" PIPE AND MULTIPLE PIPE TEST\n");
printf("=====\n");

printf("\n1. SINGLE PIPE TEST\n");
printf("-----\n");

pipe(p1);
a = fork();

if (a == 0)
{
    close(p1[0]);
    write(p1[1], msg, strlen(msg) + 1);
    close(p1[1]);
    printf("Process 1: Data sent successfully.\n");
    return 0;
}

close(p1[1]);
wait(NULL);
read(p1[0], buf, sizeof(buf));
close(p1[0]);

printf("Process 2: Data received -> %s\n", buf);
printf("Single pipe completed successfully.\n");

printf("\n2. MULTIPLE PIPE TEST\n");
printf("-----\n");

pipe(p1);
pipe(p2);

a = fork();

if (a == 0)
{
    close(p1[0]);
    close(p2[0]);
    close(p2[1]);
    write(p1[1], "Data from P1", 13);
    close(p1[1]);
    printf("Process 1: Data sent.\n");
    return 0;
}

b = fork();

if (b == 0)
{
    close(p1[1]);
    close(p2[0]);
    read(p1[0], buf, sizeof(buf));
    printf("Process 2: Received -> %s\n", buf);
    write(p2[1], buf, strlen(buf) + 1);
    close(p1[0]);
    close(p2[1]);
    return 0;
}

c = fork();

if (c == 0)
{
    close(p1[0]);
    close(p1[1]);
    close(p2[1]);
    read(p2[0], buf, sizeof(buf));

```

```

        printf("Process 3: Received -> %s\n", buf);
        close(p2[0]);
        return 0;
    }

    close(p1[0]);
    close(p1[1]);
    close(p2[0]);
    close(p2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);

    printf("All pipeline processes completed.\n");
    printf("Descriptors closed successfully.\n");

    printf("\n=====\\n");
    printf("ALL TESTS COMPLETED SUCCESSFULLY\\n");
    printf("=====\\n");

    return 0;
}

```

COMPILATION AND EXECUTION

```

gcc q12.c -o q12
./q12

```

EXECUTION OUTPUT

```
=====
PIPE AND MULTIPLE PIPE TEST
=====

1. SINGLE PIPE TEST
-----
Process 1: Data sent successfully.
Process 2: Data received -> Hello Pipeline
Single pipe completed successfully.

2. MULTIPLE PIPE TEST
-----
Process 1: Data sent.
Process 2: Received -> Data from P1
Process 3: Received -> Data from P1
All pipeline processes completed.
Descriptors closed successfully.

=====
ALL TESTS COMPLETED SUCCESSFULLY
=====
```

RESULT

Single and multiple pipe communication was implemented successfully. The program transferred data between connected processes, managed and closed file descriptors, synchronized processes using wait(), and completed the pipeline data-flow tests successfully.